

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

Bases de Datos II

Segundo Proyecto: OLAP

Cesar Fabricio Herrera Rodríguez

Catherin Madriz Badilla

Kenneth Obando Rodríguez

25/06/2026

Diagrama lógico	2
Diagrama físico	2
Microservicios y Responsabilidades.....	3
API Principal	4
Microservicio de Búsqueda	6
Servicio de Base de Datos MongoDB.....	7
Servicio de Base de Datos PostgreSQL	8
Keycloak	9
Redis.....	9
Balanceador de Carga (Nginx)	9
Pipeline CI/CD	10
Data Warehouse.....	11
DAG Airflow.....	15
Módulo de Asignación y Optimización de Rutas de	15
Entrega	15
Módulo Spark Service	16
Módulo de Dashboards.....	17
Flujo de Datos del API	19
Flujo de datos del módulo Spark-service	21

Diagrama lógico

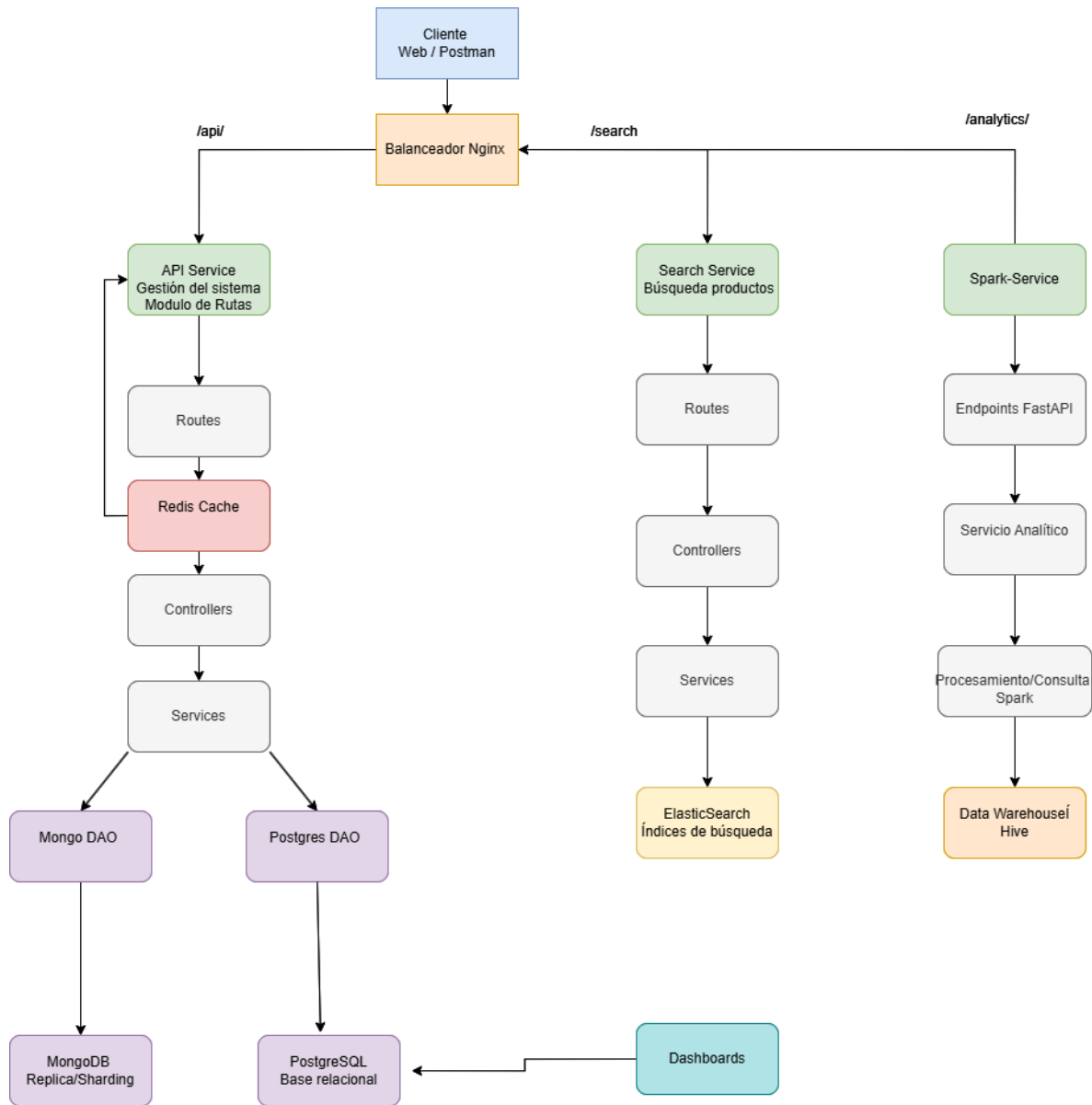
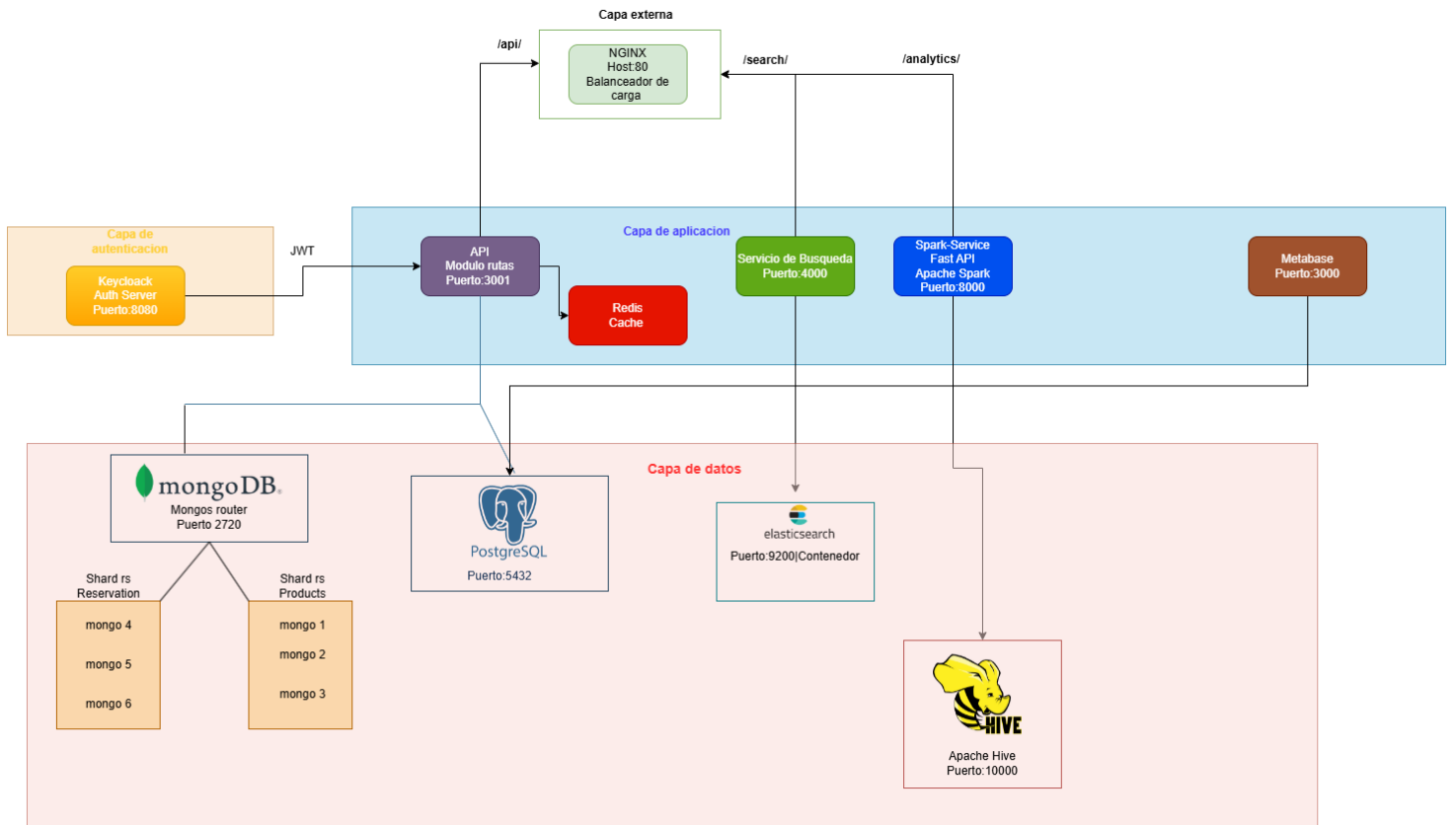


Diagrama físico



Microservicios y Responsabilidades

API Principal

La API principal es el microservicio encargado de gestionar toda la lógica de negocio del sistema de restaurantes. Este servicio expone endpoints REST para administrar menús, restaurantes, usuarios, reservas y órdenes.

Además, centraliza las validaciones del sistema y coordina la comunicación con la base de datos, Redis y el microservicio de búsqueda.

La API fue desarrollada utilizando una arquitectura por capas basada en routes, controllers, services y DAOs, permitiendo separar la lógica de negocio, el acceso a datos y el manejo de solicitudes HTTP.

La autenticación y autorización de usuarios se realiza mediante Keycloak utilizando tokens JWT.

Responsabilidades

- Gestionar menús, restaurantes, reservas, ordenes, usuarios
- Validar reglas de negocio.
- Exponer endpoints REST.
- Autenticar y autorizar usuarios mediante Keycloak.
- Acceder a PostgreSQL o MongoDB mediante el patrón DAO
- Utilizar Redis para cachear respuestas frecuentes.
- Comunicarse con el microservicio de búsqueda.
- Manejar errores y validaciones del sistema.

Endpoints

Auth

- POST /api/auth/register

Registro de un nuevo usuario.

- POST /api/auth/login

Inicio de sesión y obtención de JWT.

Users

- GET /api/users/me

Obtener detalles del usuario autenticado

- PUT /api/users/:id

Actualizar información de un usuario

- DELETE /api/users/:id

Eliminar un usuario

Menus

- POST /api/menus

Crear un nuevo menú para un restaurante

- GET /api/menus/:id

Obtener detalles de un menú específico.

- PUT /api/menus/:id

Actualizar un menú existente.

- DELETE /api/menus/:id

Eliminar un menú.

Restaurants

- POST /api/restaurants

Registrar un restaurante

- GET /api/ restaurants

Listar restaurantes disponibles.

Reservations

- POST /api/reservations

Crear una nueva reserva.

- DELETE /api/reservations/:id

Cancelar una reserva.

Orders

- POST /api /orders

Realizar un pedido.

- GET /api/orders/:id

Obtener detalles de un pedido.

- POST /api/orders/assign-drivers

Asigna automáticamente las órdenes que no tienen repartidor a los conductores activos disponibles utilizando una estrategia Round Robin,

Drivers

-GET /api/drivers/assignments/:orderId

Busca qué conductor fue asignado a una orden específica.

-POST /api/drivers

Crear un conductor.

-GET /api/drivers

Obtener todos los conductores.

- GET /api/drivers/routes

Generar y consultar las rutas optimizadas de entrega para cada conductor utilizando distancias geográficas y el algoritmo de vecino más cercano.

Microservicio de Búsqueda

El microservicio de búsqueda es el encargado de gestionar las búsquedas de productos utilizando Elasticsearch como motor de indexación y consulta.

Este servicio funciona de forma independiente de la API principal y permite realizar búsquedas textuales rápidas, filtrado por categorías y reindexación manual de productos.

La arquitectura del microservicio se encuentra separada en capas controllers, services y Daos

Responsabilidades:

- Realizar búsquedas de productos
- Filtrar productos por su categoría
- Gestionar índices en Elasticsearch
- Reindexar productos manualmente

- Procesar consultas optimizadas sobre productos indexados

Endpoints del Search-Service

- GET /search/products?q=texto

Búsqueda textual en productos.

- GET /search/products/category/:categoria

Filtrar por categoría.

- POST /search/reindex

Reindexar productos manualmente.

Servicio de Base de Datos MongoDB

El servicio de MongoDB es el componente encargado de la persistencia de datos utilizando una arquitectura distribuida basada en replicación y sharding.

En la arquitectura del proyecto se configuraron shards para las colecciones de reservas y menús, donde cada shard cuenta con su propio conjunto de réplicas compuesto por un nodo primario y dos nodos secundarios.

Responsabilidades

- Persistir información del sistema.
- Garantizar alta disponibilidad mediante replicación.
- Distribuir datos entre múltiples nodos.
- Mantener redundancia y tolerancia a fallos.

Configuración Implementada

Replica Sets

Cada shard fue configurado con:

- 1 nodo primario.
- 2 nodos secundarios.

Sharding

Se implementó particionamiento de datos para:

- Reservas.
- Menús con sus productos.

Models

- Delivey Driver
- Menu
- Reservations
- Restaurant
- User
- Order

Servicio de Base de Datos PostgreSQL

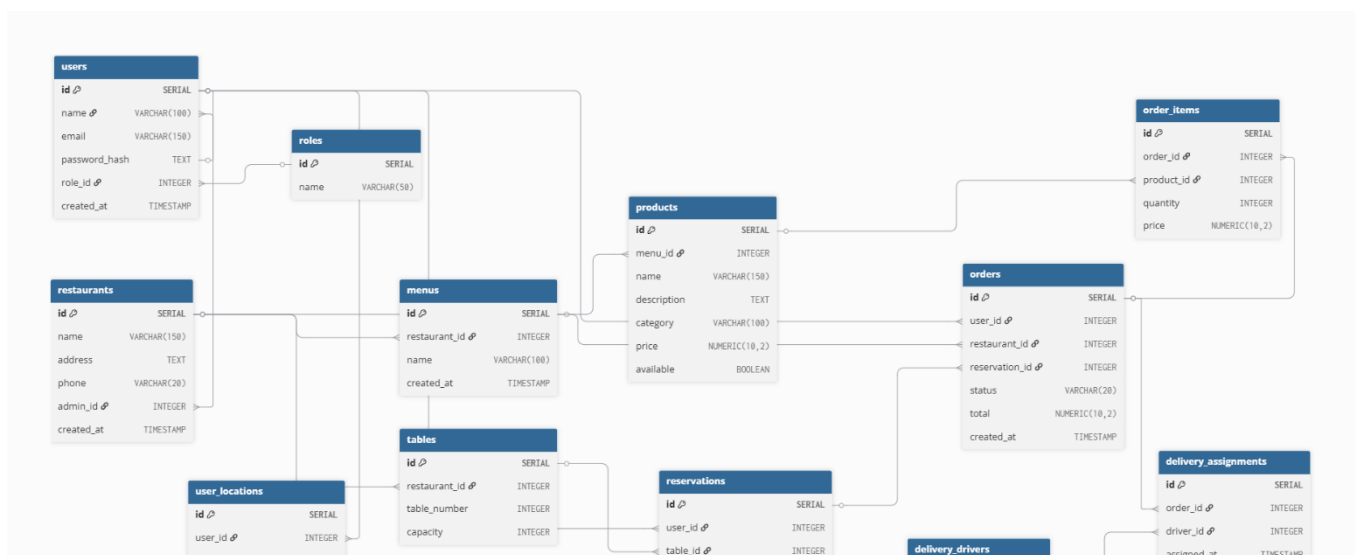
El servicio de PostgreSQL es el componente encargado de la persistencia de datos relacional dentro del sistema.

Esta base de datos se utiliza como alternativa a MongoDB y puede ser seleccionada dinámicamente mediante configuración, sin necesidad de modificar la lógica de negocio de la aplicación.

La integración fue desarrollada utilizando el patrón DAO, permitiendo desacoplar el acceso a datos de los servicios y controladores de la API principal.

Responsabilidades

- Gestionar transacciones y consistencia de datos.
- Ejecutar operaciones CRUD.
- Garantizar integridad referencial.
- Servir como alternativa configurable a MongoDB.
- Almacenar información de productos, usuarios, menús, restaurantes, reservas y órdenes.



Keycloak

Keycloak es el microservicio encargado de realizar todo el manejo y autenticación de los usuarios. Todas las rutas están protegidas por su autenticador, así como también este mismo provee un token JWT para realizar diferentes tipos de consultas HTTP

Responsabilidades:

- Autenticar a todos los usuarios
- Manejo de los datos de los usuarios, sus contraseñas y demás identificadores
- Protección de rutas, funcionando como middleware

Redis

Redis actúa como un servicio de cache para las solicitudes GET, guardando la información obtenida en estas solicitudes, haciendo que cada vez que sean llamadas, primero se busque si estas residen en Redis, en el caso de que no lo sean, almacena la respuesta utilizando el patrón CACHE-ASIDE y da un tiempo de expiración de 2 minutos a la respuesta, siguiendo estas mismas políticas de expiración para todas las rutas GET. Este microservicio funciona en las rutas, permitiendo poner la política de expiración en esta, y además verificar de una forma más fácil que las rutas que no sean GET no estén siendo cacheadas

Responsabilidades:

- Recibir solicitudes del cliente
- Actúa como middleware entre el controlador y los DAOs
- Disminuye considerablemente el tiempo de respuesta de las solicitudes
- Anular el cacheo de solicitudes que no sean GET, para evitar problemas entre solicitudes con misma ruta, pero diferente método

Balanceador de Carga (Nginx)

El balanceador de carga es el encargado de recibir todas las solicitudes que entran al sistema y redirigirlas hacia el microservicio correspondiente.

Se utilizó Nginx como punto central de acceso para los servicios del sistema. Su configuración permite definir grupos de servidores (*upstreams*) para la API principal y para el microservicio de búsqueda, facilitando la distribución del tráfico y la escalabilidad de la solución.

Responsabilidades

- Recibir solicitudes del cliente
- Redirigir tráfico hacia los microservicios correspondientes.
- Distribuir carga entre múltiples instancias de la API principal.
- Distribuir carga entre múltiples instancias del microservicio de búsqueda.
- Facilitar la escalabilidad horizontal mediante Docker Compose.

Configuración de rutas

El balanceador expone un único punto de entrada y utiliza rutas específicas para identificar el servicio destino:

- **/api/** -> Redirige las solicitudes hacia la API principal.
- **/search/** -> Redirige las solicitudes hacia el microservicio de búsqueda.
- **/analytics/** -> Redirige las solicitudes hacia el Spark Service

Grupos de servidores

Se definieron dos grupos de servidores:

- **api_cluster**: contiene las instancias de la API principal ejecutándose en el puerto 3001.
- **search_cluster**: contiene las instancias del microservicio de búsqueda ejecutándose en el puerto 4000.
- **spark_cluster**: contiene las instancias del spark-service ejecutándose en el puerto 8000.

Pipeline CI/CD

Un flujo CI/CD es un proceso automatizado que se encarga de validar, construir y preparar una aplicación cada vez que se realizan cambios en el código. Su objetivo es

asegurar que el sistema se mantenga estable, funcional y listo para ser entregado en cualquier momento. La integración continua permite detectar errores de forma temprana mediante la ejecución automática de pruebas, mientras que la entrega continua se enfoca en generar artefactos listos para su uso, como imágenes Docker, que pueden ser desplegadas posteriormente. En conjunto, este flujo mejora la calidad del software, reduce errores humanos y acelera el desarrollo.

En este proyecto, el flujo CI/CD se aplica directamente sobre el backend. Cada vez que se realiza un cambio en el repositorio, se ejecutan pruebas unitarias y de integración para validar que la lógica de negocio, la comunicación entre capas y la interacción con las bases de datos funcionen correctamente. Posteriormente, se construye una imagen Docker de la aplicación y se publica en un registry, garantizando que siempre exista una versión actualizada y lista para ser utilizada. Esto es especialmente relevante debido a la arquitectura basada en microservicios y el uso de múltiples tecnologías, lo que requiere asegurar consistencia y estabilidad en todo el sistema.

Responsabilidades

- Validar automáticamente el código ante cada cambio
- Ejecutar pruebas unitarias y de integración
- Detectar errores antes de que lleguen a producción
- Construir la aplicación de forma reproducible
- Generar imágenes Docker consistentes
- Publicar las imágenes en un registry
- Mantener versiones actualizadas del sistema
- Garantizar la estabilidad de la rama principal
- Automatizar el proceso de entrega del software
- Reducir intervención manual y errores humanos

Data Warehouse

El Data warehouse es la base principal para realizar las consultas basadas en análisis de datos, usa los contenedores de Hive, y además está basado en una arquitectura

hecha en PostgreSQL, simulando una base de datos pequeña, que contiene únicamente las dimensiones necesarias para el análisis.

Las tablas de dimensiones presentes son las siguientes

Dim_user: *userid*, *user_name*, *role*. Usada para relacionar usuarios en la tabla de hechos.

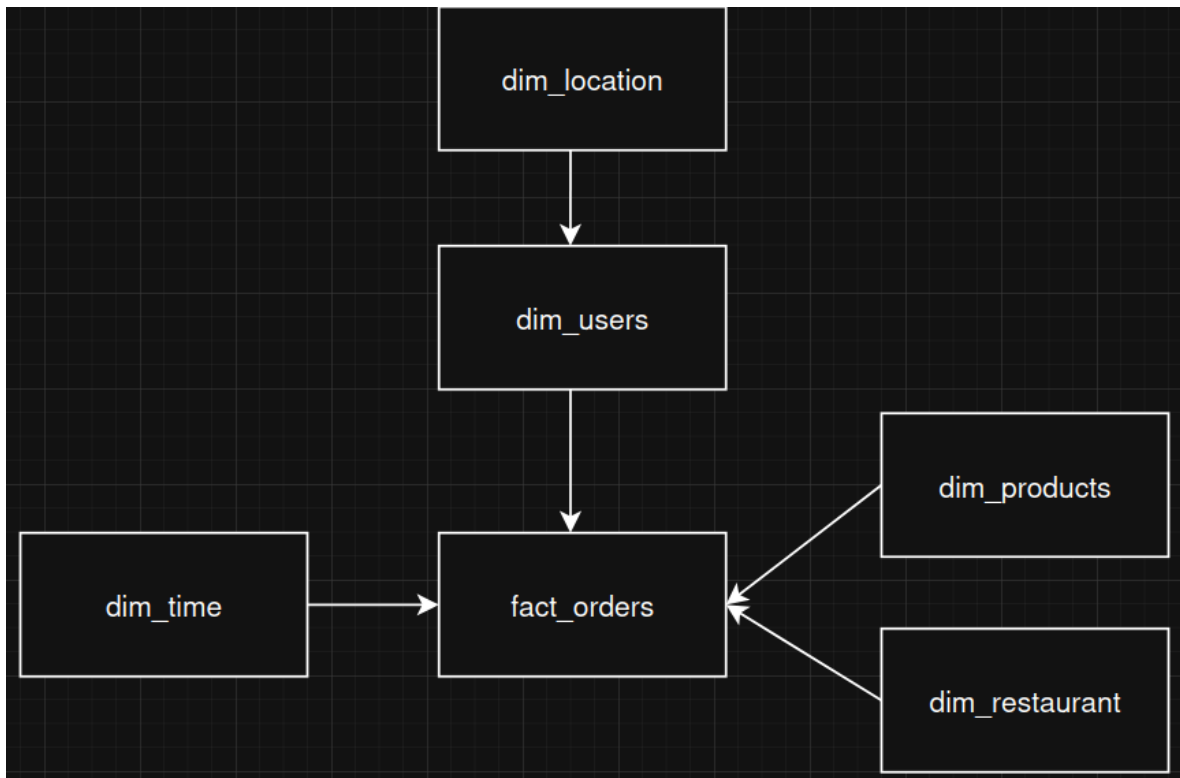
Dim_products: *product_id*, *product_name*, *price*. Usada para el cálculo de precios y análisis de comportamiento en el mercado.

Dim_restaurants: *restaurant_id*, *restaurant_name*. Identifican a restaurantes en los análisis.

Dim_time: *time_id*, *date*, *hour*. Hace referencia a la hora y fecha específica en que se realiza un pedido, útil para la generación de reportes por hora y análisis de comportamiento de los usuarios

Dim_location: *user_id*, *latitude*, *longitude*, *address*. Contiene las ubicaciones de los clientes que hacen un pedido, usado para análisis de rutas ideales

Todas estas tablas de dimensiones juntas forman la tabla de hechos, nombrada *fact_orders*, realizando un esquema de estrella de la siguiente forma



Para modificar y consultar datos del data warehouse se utiliza el módulo de spark, mas, con los siguientes comandos en la línea de comandos se puede acceder desde una CLI

```
docker exec -it hive-server bash
```

```
beeline -u jdbc:hive2://localhost:10000
```

Presentan los siguientes cubos/Vistas OLAP

1) Análisis de ventas por producto

cube_sales_by_product.product_id	cube_sales_by_product.product_name	cube_sales_by_product.price	cube_sales_by_product.total_orders	cube_sales_by_product.total_quantity_sold
1	Casado	3500.0	1	1
3	Jugo Natural	1200.0	1	1
4	Pizza Pepperoni	8000.0	1	1

desarrolla su venta y si es factible invertir más en un producto o no

2) Ventas e ingresos por tipo de producto

cube_sales_by_product_type.product_category	cube_sales_by_product_type.total_orders	cube_sales_by_product_type.total_quantity_sold	cube_sales_by_product_type.total_sales	cube_sales_by_product_type.average_sale_amount
Menú Pizzas	1	1	8000.0	8000.0
Menú Principal	2	2	4700.0	2350.0

Permite hacer un análisis más categórico de cada producto, haciendo que la transparencia por categoría sea mas accesible

3) Ventas por restaurante

1	2350.0	La Parrilla Tica	2	2	4700.0
2	8000.0	Pizza Express	1	1	8000.0

Permite ver el desempeño de cada restaurante, mostrando una media de ganancia con respecto a todos los productos vendidos por cada uno.

4) Ventas por tiempo

cube_sales_by_time.year	cube_sales_by_time.month	cube_sales_by_time.day	cube_sales_by_time.hour	cube_sales_by_time.total_orders	cube_sales_by_time.total_quantity	cube_sales_by_time.total_sales
2026	6	26	20	3	3	12700.0

Permite ver el desempeño de las ventas en general dependiendo de la hora del día, haciendo que análisis del comportamiento del mercado sean más fáciles de realizar

5) Comportamiento de los usuarios

cube_usage_frequency_by_user.user_id	cube_usage_frequency_by_user.name	cube_usage_frequency_by_user.role_name	cube_usage_frequency_by_user.total_orders	cube_usage_frequency_by_user.total_products_purchased	cube_usage_frequency_by_user.total_spent	cube_usage_frequency_by_user.average_order_value
3	Cliente 1	CLIENT	2	2	4700.0	2350.0
4	Cliente 2	CLIENT	1	1	8000.0	8000.0

Permite conocer la frecuencia y cantidad de pedidos realizados por un usuario dependiendo de su rol, hecho con el fin de analizar el mercado y que clientes pueden ser más susceptibles a anuncios, ventas espontaneas y demás

DAG Airflow

En conjunto al data warehouse, se hace un DAG con Airflow que permita que el flujo se automatice y el análisis no sea tan pesado como podría llegar a ser si se hace automáticamente con beeline todas las veces.

El DAG realiza la siguiente secuencia: Verifica que la base de datos del OLTP este viva >> Copia los datos con Spark y los transforma en dimensiones y en un archivo .csv >> Carga los datos usando beeline >> Verifica que todo se realizó de forma correcta.

Se ejecuta cada 6 horas, con el fin de hacer análisis de mercadeo en cada momento del día. Se puede hacer un análisis manual, ingresando al puerto 8081:8080, donde se mostrara el panel de administración de airflow y toda información relevante.

En el directorio sparkservice/jobs se encuentra la transformación de Spark, con la función read-postgres-table(), recopila la información de todas las tablas necesarias, luego con lo obtenido, se transforman los datos a sus dimensiones y se suben a un contenedor en docker, el cual luego se utiliza para actualizar los datos de Hive.

Módulo de Asignación y Optimización de Rutas de

Entrega

Este módulo fue desarrollado para simular el proceso de distribución de pedidos a repartidores y generar una secuencia optimizada de entregas utilizando información geográfica de los clientes.

Funcionamiento general

El módulo se divide en dos etapas principales:

1. Asignación de pedidos a repartidores

Mediante el endpoint: POST /orders/assign-drivers

El sistema consulta todas las órdenes que aún no tienen repartidor asignado y las distribuye automáticamente entre los conductores activos.

Para realizar esta distribución se utiliza una estrategia Round Robin, donde los pedidos se asignan de forma equitativa entre los repartidores disponibles.

2. Generación de rutas optimizadas

Mediante el endpoint: GET /drivers/routes

El sistema obtiene todas las órdenes asignadas a cada conductor junto con las coordenadas geográficas de entrega.

Cada pedido contiene información como: latitud, longitud, dirección

Posteriormente las órdenes se agrupan por repartidor para construir su ruta de trabajo.

Algoritmo utilizado

Para optimizar la secuencia de visitas se implementó la heurística de Vecino Más Cercano

El algoritmo funciona de la siguiente manera:

1. Selecciona un punto inicial.
2. Busca la entrega más cercana.
3. Agrega esa entrega a la ruta.
4. Repite el proceso con las entregas restantes.
5. Continúa hasta visitar todos los destinos

Cálculo de distancias

Para determinar qué entrega es la más cercana se utiliza la fórmula de Haversine.

La fórmula calcula la distancia real entre dos puntos geográficos utilizando:

- Latitud
- Longitud
- Radio terrestre

Implementada mediante el método: calculateDistance()

El resultado se obtiene en kilómetros.

Módulo Spark Service

El módulo Spark Service se encarga de realizar consultas analíticas sobre el Data Warehouse almacenado en Hive utilizando Apache Spark. Su propósito es procesar

grandes volúmenes de datos de forma eficiente y generar información útil para la toma de decisiones.

La solución fue desarrollada utilizando:

- Apache Spark para el procesamiento analítico.
- Apache Hive como almacén de datos.
- FastAPI para exponer los resultados mediante servicios REST.

Análisis 1: Tendencias de Consumo

Este análisis identifica cuáles productos son los más vendidos dentro del sistema.

Para ello se utilizan principalmente:

- fact_orders
- dim_product

El resultado muestra: Nombre del producto y Cantidad total vendida.

Endpoint: GET /analytics/consumption

Análisis 2: Horarios Pico

Este análisis determina las horas del día en las que se generan más pedidos.

Utiliza principalmente:

- fact_orders

El resultado muestra: Hora del día y Cantidad de pedidos registrados.

Endpoint: GET /analytics/peak-hours

Análisis 3: Crecimiento Mensual

Este análisis muestra la evolución de las ventas a lo largo del tiempo.

Utiliza principalmente:

- fact_orders
- dim_time

El resultado incluye:

- Año.
- Mes.
- Total de ventas.
- Cantidad de pedidos.

Endpoint : GET /analytics/monthly-growth

Los resultados de los análisis generan un .csv que queda almacenado en la carpeta outputs

Módulo de Dashboards

Este módulo permite visualizar información importante del sistema mediante gráficos e indicadores obtenidos directamente desde la base de datos. Su objetivo es facilitar el análisis de los datos generados por los restaurantes, clientes y pedidos.

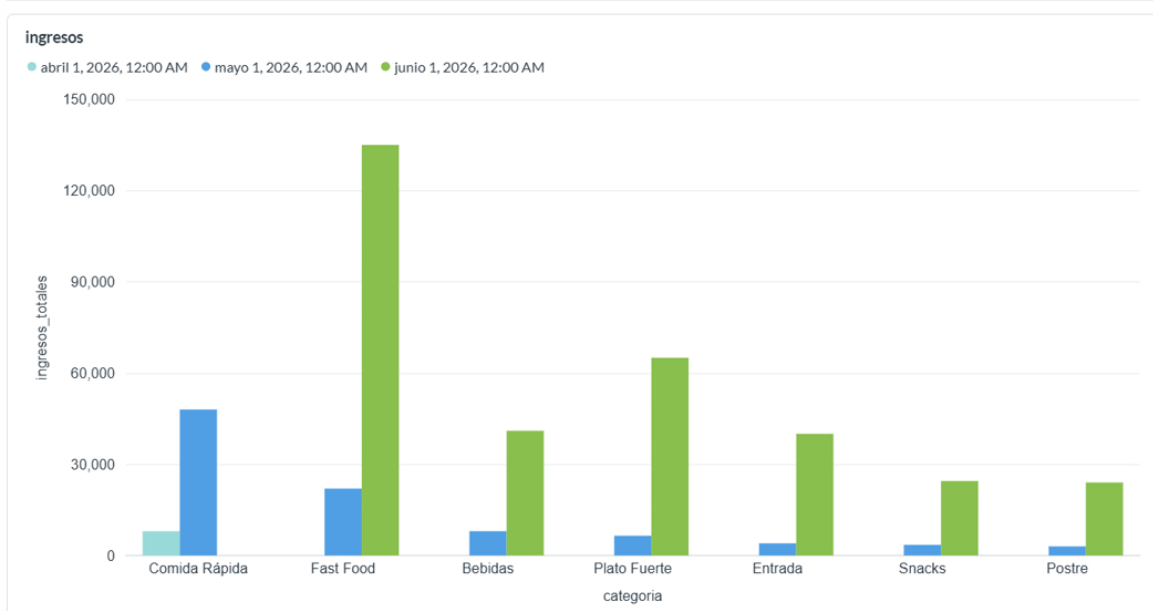
Para la implementación se utilizó Metabase, una herramienta de visualización de datos que permite crear dashboards interactivos a partir de consultas SQL.

Se desarrollaron tres dashboards principales:

Ingresos por mes y categoría de producto

Muestra cuánto dinero generan los productos vendidos, agrupando la información por mes y por categoría. Esto permite identificar cuáles categorías venden más.

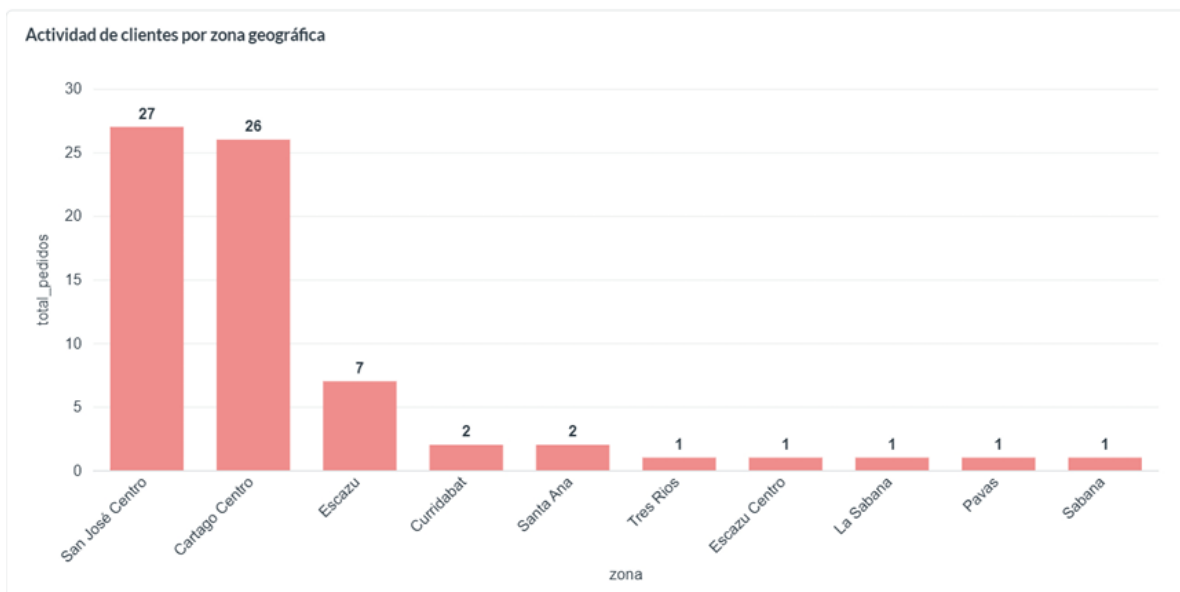
Dashboard 1 - Ingresos



Actividad de clientes por zona geográfica

Presenta la actividad de los clientes según la ubicación registrada. Permite observar cuáles zonas generan más pedidos

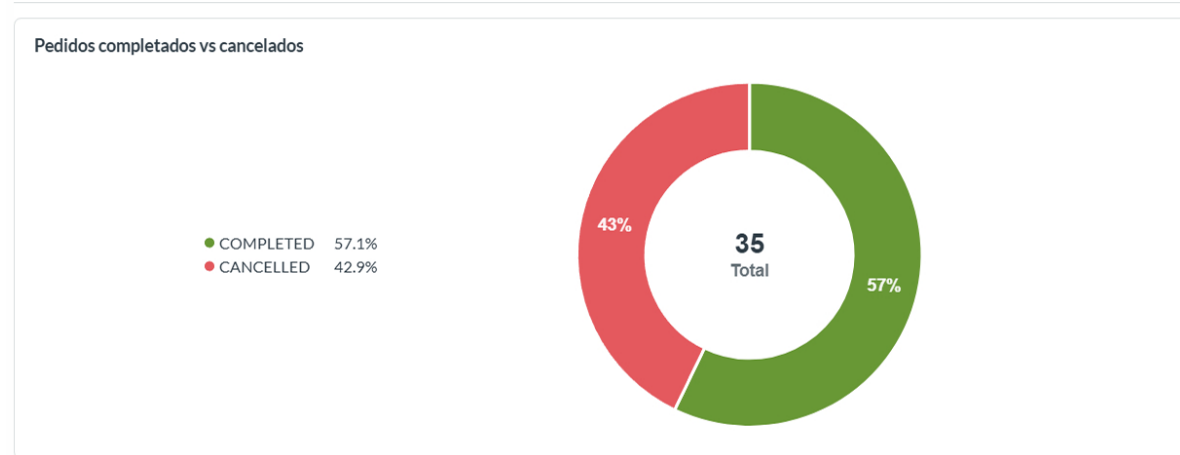
Actividad de clientes por zona geográfica



Estadísticas de pedidos completados vs cancelados

Muestra la distribución de los pedidos según su estado. Con esta información es posible conocer cuántos pedidos fueron completados, cancelados

Pedidos completados vs cancelados



Los dashboards se alimentan mediante consultas SQL almacenadas en la carpeta dashboards/sql, mientras que las exportaciones generadas desde Metabase pueden almacenarse en la carpeta dashboards/exports.

Flujo de Datos del API

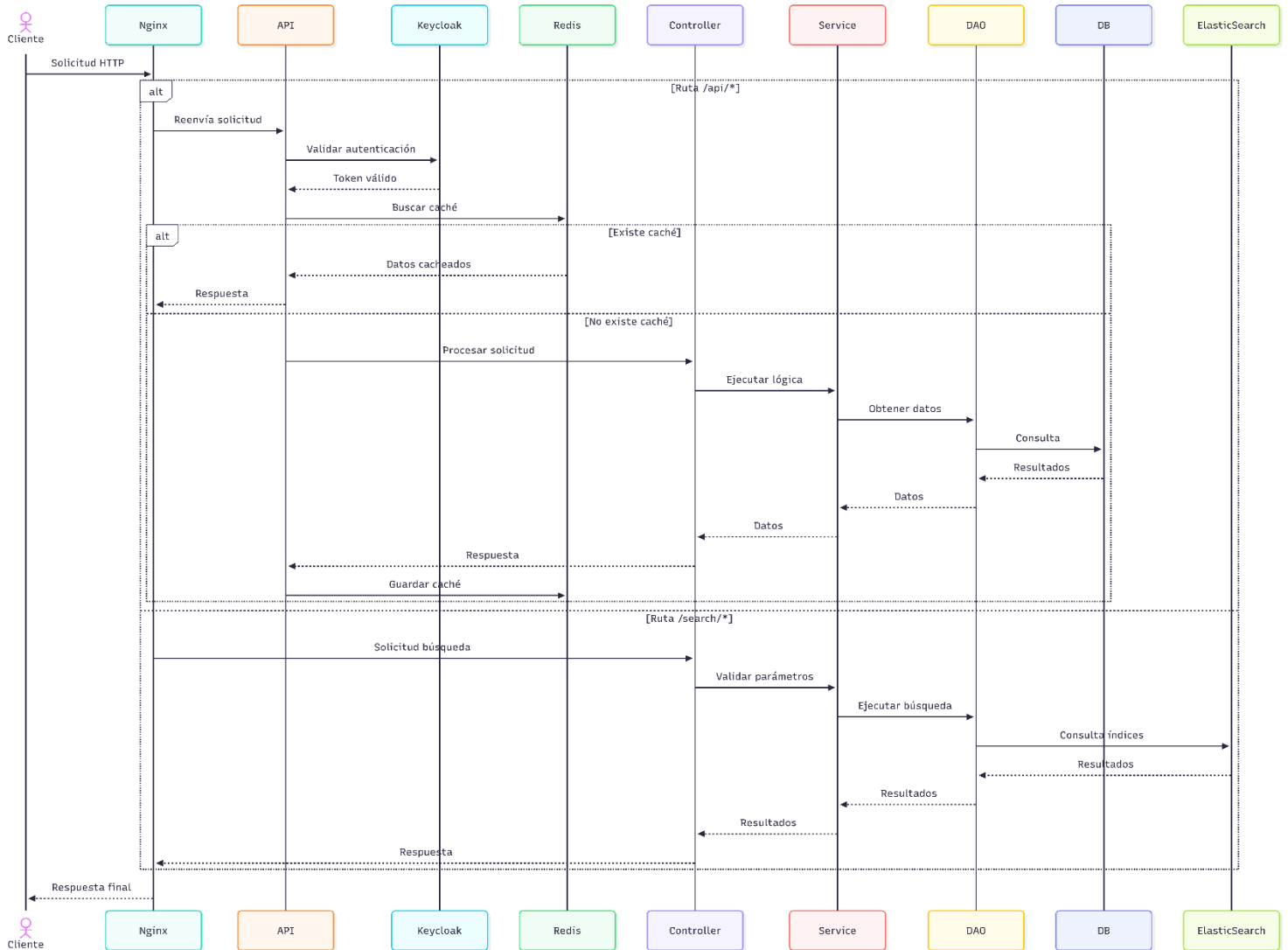
El sistema utiliza una arquitectura basada en microservicios donde todas las solicitudes ingresan inicialmente a través del balanceador de carga Nginx.

Nginx actúa como punto de entrada principal y se encarga de redirigir las solicitudes hacia el servicio correspondiente según la ruta solicitada.

Flujo General de Solicitudes

1. El cliente realiza una solicitud HTTP al sistema.
2. Nginx recibe la solicitud y analiza la ruta:
 - Las rutas `/api/` son dirigidas hacia la API principal.
 - Las rutas `/search/` son dirigidas hacia el microservicio de búsqueda.
3. Cuando la solicitud corresponde a la API principal:
 - La API valida autenticación mediante Keycloak.
 - Se consulta Redis para verificar si existe información cacheada.
 - Si la información existe en caché, la respuesta se retorna directamente.
 - Si no existe caché, la API continua por las diferentes capas de controller y services hasta llegar a la capa de DAO donde se obtienen los datos
 - La respuesta obtenida puede almacenarse nuevamente en Redis si no lo estaba antes.
4. Cuando la solicitud corresponde al microservicio de búsqueda:
 - Las rutas del servicio reciben la solicitud y la dirigen hacia el controller correspondiente.
 - El controller valida los parámetros básicos de la búsqueda y delega la operación al service.
 - El service utiliza la capa DAO para interactuar con ElasticSearch.
 - El DAO ejecuta las consultas sobre los índices de productos almacenados en ElasticSearch.
 - Los resultados obtenidos son retornados desde el DAO hacia el service.
 - En operaciones de reindexación, el microservicio obtiene nuevamente los productos desde la API y actualiza los índices almacenados en ElasticSearch.
5. Finalmente, la respuesta regresa al cliente a través de Nginx.

Diagrama flujo de datos



Su función principal es tomar los datos almacenados en el Data warehouse de Hive, procesarlos mediante Apache Spark y poner los resultados a disposición mediante servicios web.

Cuando un usuario solicita un análisis, la petición llega a un endpoint de la API desarrollado con FastAPI. Este endpoint invoca la lógica correspondiente dentro del servicio analítico.

A continuación, el servicio utiliza Apache Spark para consultar las tablas almacenadas en Hive. Spark recupera los datos necesarios y realiza los cálculos requeridos para generar indicadores como productos más vendidos, horarios con mayor cantidad de pedidos o evolución de las ventas por mes.

Una vez finalizado el procesamiento, los resultados son transformados a formato JSON y enviados nuevamente al cliente a través de la API.

Diagrama flujo de datos general

